ERIS Review Demo Guide

How to run and present the ERIS analyst console · BCSE497J Project-I, Review 3

1. Start the demo (2 minutes before the examiners)

On the demo MacBook, open Terminal and run:

```
cd ~/Desktop/eris
./run_demo.sh --reset
```

The browser opens **http://localhost:8000** by itself. Before you start, check:

- The top bar shows **T5 · 45.24M params · results_live/model** with a green dot and **LIVE**.
- There is **no** yellow "development model" pill.
- Leave the stream running for about a minute so the traffic chart fills up.
- No internet is needed: the model, the data and the charts are all local.

To stop the server, press **Ctrl+C** in Terminal. **--reset** clears old alerts, so you start with an empty queue.

2. The four pages

Page	What it shows
Live Monitor	Held-out CICIoV2024 test frames replayed as live CAN traffic. Each frame is classified by T5 in real time. Includes KPIs, a traffic chart, the live CAN bus ticker, active incidents and attack-injection buttons.
Alerts	The analyst queue. You can filter, search, bulk-acknowledge or resolve, and assign alerts. Clicking an alert opens its **explanation**, its **risk assessment** and its audit trail.
Incidents	Alerts grouped by attack class and CAN ID. Each incident has a status workflow and notes, and incidents can be exported as a CSV or JSON report.
Model Stats	Every result from the report (Ch. 5 and 7): metrics, loss curve, confusion matrix, per-class and pattern-level tables, comparison with related work, and a browser for all 1,800 test predictions. A toggle switches between the reported run and the live re-run on this laptop.

3. Talk track (about 8 minutes)

#	Page	Click	Say
1	Live Monitor	Point to the ticker and the chart.	ERIS replays the **held-out** test frames of CICIoV2024 as live CAN traffic, so the model has never seen them. Every frame goes through three stages: classification by T5, then explanation, then risk.
2	Live Monitor	Click **Steering spoofing** under "Inject".	An attack burst hits the bus, red frames appear in the ticker, and a **New Critical incident** pop-up shows up.
3	Alerts	Click the top **Critical** alert.	This is the analyst queue: severity, class, CAN ID, confidence and status.
4	Alert drawer	Scroll slowly through it.	**Explainability is our contribution.** The heatmap shows which fields drove the decision, and the rationale explains it in plain English. Also point out the **Identifier vs payload** card (which part decides), the **counterfactual** (the smallest change that makes the frame benign, re-checked by the model), and the method-agreement and IG completeness checks. Finally, the risk card: the affected vehicle function, impact x feasibility (ISO/SAE 21434) and the recommended actions.
5	Alert drawer	Click **Acknowledged**. Type a name and click **Assign**. Add a note.	Every analyst action is logged with who did it and when.
6	Incidents	Open the incident, click **Resolved**, then **Export CSV**.	Thousands of repeated frames become one incident, so the analyst isn't flooded. The export produces an incident report.

#	Page	Click	Say
7	Model Stats	Scroll from the top of the page.	These are all the numbers from Chapter 7, so nobody has to read terminal output: 91.72% row accuracy, macro F1 0.9118, 97.0% pattern-level accuracy (32 of 33 frames), and the reload check. Then switch to **Live re-run**: we retrained on this laptop and got exactly the same results.
8	Model Stats	In **Error analysis**, click **Explain this frame**.	This is the single error from Section 7.4. With the CAN ID hidden, ERIS still says SPEED. With the payload hidden, it says RPM. So the SPEED-identical payload causes the error, which the explanation confirms with evidence.

Suggested split for three presenters: steps 1-2 (intro and live detection), steps 3-6 (explainability and alert management), steps 7-8 (evaluation and error analysis).

4. Likely questions

Question	Answer
Is this live or pre-recorded?	Live. T5 classifies every frame at request time and explanations are computed on demand. Only the input traffic is replayed, from held-out test frames.
Why replay instead of a real car?	The scope says ERIS works offline on recorded traffic (Section 1.3). Only test patterns the model never saw are replayed.
Why 91.7% and not 99%?	Results of around 99% come from duplicated rows leaking across the train/test split. We split by unique frame (Model Stats, Table 7.4), so our score is honest.
How do you know the explanation is faithful?	Integrated Gradients satisfies completeness: its attributions add up to the change in the model's output, and we show that check. Two independent methods are compared (Spearman rho). The counterfactual is re-classified by the model to confirm it.
What does the model rely on?	Mostly the CAN ID. Across the 8 attack test frames: payload decides 3, the ID decides 2, either one alone suffices for 2, and 1 needs both. This is the shortcut risk listed in Limitations (Section 7.6), and explainability is how ERIS makes it visible.
How is risk computed?	Impact x attack feasibility per vehicle function (ISO/SAE 21434 style). It is downgraded when the model's confidence is low and escalated when an attack is sustained.

5. If something goes wrong

Problem	Fix
"development model" pill, or "recorded predictions" in the top bar	results_live/model is missing. Run `./train_live.sh` (about 12 min), then start `./run_demo.sh` again.
Port 8000 already in use	`PORT=8001 ./run_demo.sh`, then open localhost:8001.
Too many alerts on screen	Stop with Ctrl+C and restart with `./run_demo.sh --reset`.
Page looks stuck	Refresh the browser (Cmd+R). The server keeps all alerts.
Running on another laptop	Use `git checkout demo-dashboard`, create the venv (see README), copy the `data/` and `results_live/model/` folders across, then run `./run_demo.sh`.